

Programmazione e calcolo scientifico

Esercitazione 2 - Indirizzi, primi algoritmi

18 marzo 2026

1 Posizionamento in memoria degli oggetti

Si consideri il programma proposto in Figura 1 e si stampino gli indirizzi degli elementi degli array e delle variabili `x` e `y`, per esempio con

```
std::cout << &ad[2] << "\n";
```

per quanto riguarda gli array e con

```
std::cout << &x << "\n";
```

per quanto riguarda le variabili scalari. Si fornisca quindi una rappresentazione grafica di come gli array e le variabili sono posizionati in memoria, facendo attenzione alla posizione relativa dei vari oggetti. Successivamente, si compili e si esegua il programma così com'è dato in Figura 1, aggiungendo soltanto la stampa degli indirizzi di `x` e `y`. Se l'istruzione a riga 14 stampa uno zero, provare a spiegarne il perché. Annotare le osservazioni e aggiungere il materiale prodotto al proprio repository Git (può essere anche una scansione della rappresentazione grafica e delle osservazioni scritte a mano).

Nota: il programma, una volta arrivato a riga 14, potrebbe non stampare lo zero ma stampare "1". In tal caso, se si è compilato il programma con `gcc` provare a ricompilarlo con `clang` (o viceversa, usare `gcc` se inizialmente si era usato `clang`). Si osservi quindi il risultato ottenuto con i diversi compilatori: se ancora non si riesce ad ottenere la stampa dello zero si provi comunque a spiegare il significato dell'istruzione a riga 12.

```

1  #include <iostream>
2
3  int main()
4  {
5      double ad[4] = {0.0, 1.1, 2.2, 3.3};
6      float af[8] = {0.0, 1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7};
7      int ai[3] = {0, 1, 2};
8
9      int x = 1;
10     float y = 1.1;
11
12     (&y)[1] = 0;
13
14     std::cout << x << "\n";
15
16     return 0;
17 }

```

Figura 1: Esercizio

2 Primi algoritmi

Si definisca un array di `double` di lunghezza `N` e lo si riempia con valori a piacere. Se ne calcoli minimo, massimo, media e deviazione standard. Stampare i valori calcolati.

Nel calcolo delle quantità richieste è utile utilizzare le funzioni `std::min()`, `std::max()` e `std::sqrt()` fornite dalla libreria standard. Cercare nel sito <https://en.cppreference.com> la documentazione di tali funzioni.

Si ricordi che la lunghezza `N` dell'array che si va a dichiarare **deve essere nota a tempo di compilazione**. Si dichiara quindi l'array usando del codice come

```

1  static const int N = 10;
2  double arr[N];

```

e non del codice come

```

1  int N = 10;
2  double arr[N];

```

In questo ultimo caso si finirebbe per dichiarare un *variable length array*, che è una funzionalità del linguaggio assolutamente da evitare se non si conoscono i dettagli del suo funzionamento. Per evitare di dichiarare involontariamente dei variable length arrays, aggiungere l'opzione `-Werror=vla` alla chiamata al compilatore affinché venga emesso un errore. Per esempio:

```
g++ -Werror=vla -o es2 es2.cpp
```

Si fornisca un opportuno `CMakeLists.txt` per il programma, che includa le opzioni relative all'attivazione degli errori in caso d'uso di un VLA. Si attivino anche tutti i warning con `-Wall` ed in particolare i warning relativi al shadowing con `-Wshadow`.

```
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -Wshadow -Werror=vla")
```

2.1 Algoritmo Bubblesort (facoltativo)

Più avanti nel corso si vedranno gli algoritmi di ordinamento, utili per ordinare sequenze di elementi di un dato tipo. Uno dei più semplici algoritmi di ordinamento è chiamato *Bubblesort* e, volendo ordinare un array, funziona nel seguente modo:

1. Si scorra tutto l'array da ordinare, confrontando gli elementi adiacenti. Se l'elemento in posizione i -esima è più grande dell'elemento in posizione $(i + 1)$ -esima, si scambino i due elementi,
2. Se nel passo precedente sono stati effettuati uno o più scambi, ritornare al punto 1. Se invece non è stato effettuato nessuno scambio l'algoritmo termina.

Provare ad implementare l'algoritmo sopra descritto e ad applicarlo all'array precedentemente definito.

3 Modalità di svolgimento e consegna

Dev'essere consegnato quanto richiesto più sopra, includendo un `CMakeLists.txt` che automatizzi la compilazione dei programmi. I file devono essere posizionati in una cartella chiamata **esercitazione2** nel proprio repository.

La consegna delle esercitazioni è obbligatoria e da effettuarsi entro i termini prescritti. Il tempo assegnato per lo svolgimento di questa esercitazione è di una settimana. Termine ultimo per la consegna di questa esercitazione sono le ore 23.59 di Mercoledì 25/03/2025: fa fede il timestamp del commit taggato `consegna_es2` sul proprio repository. Eventuali giustificazioni vanno presentate a `matteo.cicuttin@polito.it` entro il termine ultimo previsto per la consegna dell'esercitazione.

Si ricordi che l'uso di strumenti di intelligenza artificiale è scoraggiato ed in ogni caso dev'essere dichiarato esplicitamente all'interno dei sorgenti consegnati.

All'esame potrebbe essere richiesto di mostrare il funzionamento del codice, pertanto ci si assicuri che il codice nel proprio repository sia funzionante.